Synthetic Data Governance for Regulated Healthcare (HIPAA/HITRUST + Enterprise Data Governance)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this is . . . 4
- The governance model . . . 5
- How it works (the deployment model) . . . 5
- The target model . . . 6
- Benefits . . . 6
- Limitations (read these) . . . 6

Requirements 7

- R1. A declared control set (the standards to align to) . . . 7
- R2. A synthetic-asset classification scheme . . . 7
- R3. Named governance roles (owner / steward / reviewer / consumer) . . . 8
- R4. A catalog store location (governed, versioned) . . . 8
- R5. An inventory hook into where synthetic/test assets live . . . 8

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

- 1. A Python data/engineering environment . . . 9
- 2. A target clinical data model / standard . . . 9
- 3. An isolated non-production test environment . . . 10
- 4. Privacy, governance & compliance stakeholders and policy . . . 10
- 5. AI/QA use cases and a pilot scope . . . 10
- 6. Reference/real-data access for fidelity (read-only, minimized) . . . 10

Implementation 11

- Phase 0 — Confirm prerequisites . . . 11
- Phase 1 — Assess (establish the readiness baseline) . . . 11
- Phase 2 — Design (policy, scheme, roles, control mapping) . . . 12
- Phase 3 — Generate/Build (catalog, classify, approve, access) . . . 12
- Phase 4 — Validate (run the governance gates on the pilot) . . . 13
- Phase 5 — Expand (widen coverage, carry the gates) . . . 13
- Rollback (applies at every gate) . . . 14

Testing & Validation 14

- T1 — Readiness baseline is produced (Gate G1) . . . 14
- T2 — Design artifacts exist and are reviewed (Gate G2) . . . 14
- T3 — Every pilot asset has lineage (catalog completeness) . . . 15
- T4 — Every pilot asset is classified (classification completeness) . . . 15
- T5 — Approval is enforced before shared use (approval integrity) . . . 15

T6 — Access is aligned to classification (access model) . . . 15
T7 — Control evidence view is complete and honest (control mapping) . . . 15
T8 — Rollback works . . . 16
Overall pilot pass criteria . . . 16
Troubleshooting 16
Assess . . . 16
Design . . . 17
Build . . . 17
Validate . . . 18
Operate / Expand . . . 19
Best Practices 19
Governance discipline . . . 19
Keep the boundary bright . . . 20
Registers & evidence . . . 20
Process & scale . . . 20
Deliverables Summary 21
Documentation . . . 21
The 5 Python tools . . . 21
Register artifacts (you produce these) . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

Outcome. Synthetic and test data managed as **governed, auditable assets** — cataloged with lineage, classified, approved before shared use, and access-controlled — aligned to the organization's HIPAA/HITRUST and enterprise data-governance obligations. The synthetic-data capability scales across service lines without accumulating shadow data risk.

The problem. In a regulated healthcare enterprise, synthetic and test datasets multiply quickly: generated, forked, copied into AI sandboxes, and shared between QA teams. Left ungoverned they become a shadow risk — no recorded origin, no classification, no approval, no evidence of alignment to the standards the organization is held to. Even artificial data needs governance when it lives inside a regulated estate.

The approach. This solution establishes six connected governance components — a policy and roles, a catalog with lineage, a classification scheme, an enforced approval workflow, an access model, and a control mapping to the organization's declared control set. It is delivered as a self-guided method plus five Python tools, and follows a pilot-first **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model with explicit pass/fail gates (lineage → classification → approval → access → control evidence) and a defined rollback. Prove it on one service line, then expand.

Why it is safe. The governance registers are **metadata-only** and never hold real Protected Health Information; the category rule — never real PHI into a shared harness, twin, or AI test environment — is preserved throughout. Every tool is safe-by-default (dry-run where it writes) and standard-library-first with no invented network calls. Critically, the solution draws a **bright line**: it delivers governance **method and evidence** — the catalog, classification records, approval decisions, and control-alignment view — and **defers the determinations** (HIPAA/HITRUST de-identification adequacy, disclosure-risk acceptance, IRB/privacy sign-off, control sufficiency) to the organization's privacy, compliance, and legal functions. Green tool output is evidence for those functions to interpret, never a compliance sign-off.

The bottom line. Synthetic/test data becomes a set of governed, cataloged, classified, approved, and access-controlled assets aligned to HIPAA/HITRUST — so the capability scales safely — at a fraction of the cost of a bespoke consulting engagement.

Solution Overview

What this is

Synthetic and test data does not escape governance just because it is artificial. In a regulated healthcare enterprise, an ungoverned pile of "test" datasets — copied, forked, shared into AI sandboxes, and forgotten — becomes a **shadow data risk**: nobody knows where a given dataset came from, what it was derived from, who classified it, who approved it for use, or whether it aligns to the organization's HIPAA/HITRUST and enterprise data-governance obligations.

This solution establishes **synthetic data governance**: the policy, catalog and lineage register, classification scheme, approval workflow, access model, and control set that make synthetic and test data **managed, auditable, and aligned** to the regulated-data constraints the organization already lives under — so the synthetic-data capability can scale without becoming an unmanaged liability.

It delivers a **governance method plus evidence**, not compliance determinations. The catalog, classification records, approval decisions, and control-mapping this solution produces are the technical

artifacts and evidence your privacy, compliance, and legal functions consume. Whether a given synthetic asset meets a HIPAA de-identification standard, whether a residual disclosure risk is acceptable, and whether a HITRUST control is satisfied are **determinations reserved to those functions** — this solution never makes them for you.

The governance model

Six connected components, each of which the implementation stands up in order:

1. **Policy & roles** — a written Synthetic Data Governance Policy that states scope (what counts as a governed synthetic/test asset), the roles (data owner, steward, privacy/compliance reviewer, consumer), the classification scheme, and the rules for approval, access, retention, and disposal. Roles map to your existing RACI where one exists.

1. **Catalog & lineage** — a register of every governed synthetic/test asset, recording where it came from (generator, source model, reference statistics used), what it was derived from (parent asset), and where it flows (which AI/QA environments consume it). Lineage is the spine: an asset with no recorded origin is a finding.

1. **Classification** — every asset carries a classification (e.g. `synthetic-only`, `derived-from-real-stats`, `contains-minimized-real`) that drives its handling, access, and the controls that apply. Classification is a stewarded, recorded decision — not an attribute the generator sets.

1. **Approval workflow** — no governed asset is released to a shared AI/QA environment until it has an **approval record**: a stated requester, purpose, classification, the reviewer(s), the decision, and the date. Approval is enforced, evidenced, and revocable.

1. **Access model** — least-privilege access to assets by classification and environment, so the right consumers reach the right assets and no one silently copies a higher-classification asset into a lower-controlled sandbox.

1. **Controls & alignment** — the governance controls (the ones this solution operates) are mapped to the organization's declared control set (e.g. selected HIPAA Security Rule safeguards and HITRUST CSF control references the compliance team provides), producing an evidence view of which governance controls are in place, partial, or absent — for the compliance function to interpret.

How it works (the deployment model)

Like every solution in this category, the work follows a pilot-first **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model with explicit pass/fail gates and a documented rollback:

• **Assess** — profile the current state: what synthetic/test assets already exist, which have any lineage, what policy and roles exist, and which declared controls are already met. Produce a **Synthetic Data Governance – Enterprise – Readiness** baseline.

• **Design** — author the **Synthetic Data Governance Policy – Enterprise – v1**, the classification scheme, the role assignments, and the control mapping against the declared control set.

• **Generate/Build** — stand up the **Synthetic Asset Catalog – Enterprise – v1** (a JSON-backed

register), register the pilot's assets with lineage, classify them, and wire the approval + access records.

- **Validate** — run the governance gates on a named pilot service line: is every pilot asset cataloged with lineage, classified, and approved before use; is access aligned; are the declared controls evidenced? Each gate has an explicit pass/fail.
- **Expand** — only after the pilot passes, extend the catalog, classification, and approval workflow to additional service lines and asset types, carrying the same gates forward.

A **rollback** is always defined: if a gate fails, un-approved or misclassified assets are pulled from shared environments and the release is held until the finding is resolved.

The target model

This solution governs synthetic/test **assets** regardless of the clinical model they represent (FHIR bundles, OMOP CDM extracts, or a source system's native schema). The catalog records each asset's target model as metadata so governance spans a heterogeneous estate. The governance artifacts themselves are JSON-backed and human-readable, so they integrate with an existing enterprise data catalog where one exists, or stand alone where one does not.

Benefits

- **Lineage you can audit** — every governed asset has a recorded origin and derivation; "where did this dataset come from?" has an answer.
- **Enforced approval** — assets reach shared AI/QA environments only with an evidenced approval, killing the shadow-copy pattern.
- **Alignment evidence** — a clear, current view of which governance controls map to the declared HIPAA/HITRUST + enterprise control set are in place, partial, or absent — for compliance to interpret.
- **Scales safely** — a repeatable, pilot-proven method so adding service lines and asset types does not multiply risk.
- **Self-guided** — exact steps and Python tooling a competent data/platform engineer can run, at a fraction of the cost of a bespoke consulting engagement.

Limitations (read these)

- **Not a compliance determination.** This solution produces governance method and evidence. HIPAA/HITRUST de-identification determinations, disclosure-risk acceptance, and IRB/privacy-office sign-off are **deferred to your privacy, compliance, and legal functions**.
- **Declared control set is an input.** The control mapping is only as good as the control set your compliance team declares; this solution does not decide which controls apply.
- **No real PHI in shared environments.** Governance records describe assets; the assets themselves follow the category rule — never real PHI into a shared harness, twin, or AI test environment.

- **Tooling is pattern-level.** Where the solution references an enterprise catalog, generator, or

AI framework, it does so at the capability level; verify version-specific behavior against your vendor's documentation.

Requirements

These are the **solution-specific** prerequisites for standing up synthetic data governance. They build **on** the six category-wide requirements in `global-requirements-synthetic.md` (a Python data/engineering environment, a target clinical data model, an isolated non-production environment, privacy/governance/compliance stakeholders and policy, AI/QA use cases and a pilot scope, and read-only minimized reference access). Verify those first — this file does not repeat them.

The governance work also inherits the category **safety/vendor/regulatory boundary**: never real PHI into a shared harness/twin/AI test environment; vendor tooling is referenced at the pattern level; and HIPAA/HITRUST de-identification, IRB/privacy sign-off, and disclosure-risk acceptance are **deferred** to the organization's privacy, compliance, and legal functions.

R1. A declared control set (the standards to align to)

- **Description:** A written, versioned list of the specific controls the synthetic-data capability

must align to — the selected HIPAA Security Rule safeguards, the applicable HITRUST CSF control references, and any enterprise data-governance control identifiers — each with a stable ID and a short statement. This is the **input** the control mapping is measured against.

- **Why required:** The compliance checker (Tool 4) maps the governance controls this solution

operates to *your* declared control set. Without a declared set there is nothing to align to and no evidence view to produce.

- **How to verify:** Confirm with the compliance/privacy function that a versioned control set

exists, that each control has an ID and statement, and that it is the authoritative list for this capability. Save it as `Declared Control Set – Enterprise – v1`.

- **How to implement if missing:** Ask the compliance function to declare the in-scope controls

(they decide which apply — this solution does not). Capture them in the control-set JSON the checker consumes. Do **not** invent controls or their applicability.

R2. A synthetic-asset classification scheme

- **Description:** A small, explicit set of classification values for synthetic/test assets — for

example `synthetic-only` (no real data used), `derived-from-real-stats` (built from governed reference statistics), and `contains-minimized-real` (read-only minimized real data under the privacy controls) — each with a handling rule (allowed environments, access, retention).

- **Why required:** Classification drives access, approval scrutiny, and which controls apply. The

catalog and approval tools record it per asset; without an agreed scheme classifications are inconsistent and ungovernable.

• **How to verify:** Confirm the scheme is documented, that each value has a handling rule, and that the privacy function has reviewed it. Save it as `Classification Scheme – Enterprise – v1`.

• **How to implement if missing:** Draft the scheme with the data-governance and privacy stakeholders during **Design**; keep it minimal (3–5 values). The classification of any specific asset is a stewarded decision, not an automated one.

R3. Named governance roles (owner / steward / reviewer / consumer)

• **Description:** The people (or role assignments) who will operate the governance: a **data owner **accountable for each asset domain, a** steward **who catalogs and classifies, a** privacy/compliance reviewer **who approves releases, and the** consumers** (AI/QA teams) who request assets. Map to the existing RACI where one exists.

• **Why required:** Approval and classification are recorded human decisions; they need named, accountable roles. An approval record with no reviewer is not an approval.

• **How to verify:** Confirm each role has a named holder (or group) and that reviewers have the authority to approve releases. Save as `Governance Roles – Enterprise – v1`.

• **How to implement if missing:** Assign the roles during **Design**, reusing existing data-governance roles where possible. A single person may hold multiple roles in a small org, but requester and approver must not be the same person on a given release.

R4. A catalog store location (governed, versioned)

• **Description:** A governed, access-controlled, version-controlled location for the JSON-backed governance registers (catalog, classification/approval log, control mapping) — for example a restricted repository or the metadata layer of an existing enterprise data catalog. It holds **metadata only**, never the asset payloads and never real PHI.

• **Why required:** The registers are the evidence of governance. They must be durable, auditable (change-tracked), and access-controlled so approval records cannot be silently altered.

• **How to verify:** Confirm the location exists, is access-controlled and version-tracked, and that the steward can write to it while consumers can read the parts they need. Save as `Governance Register Store – Enterprise – v1`.

• **How to implement if missing:** Provision a restricted, version-controlled location before registering the first asset. The Python tools read/write JSON files that live here.

R5. An inventory hook into where synthetic/test assets live

• **Description:** A read path to the environments and storage where synthetic/test assets actually exist today (sandbox buckets, test databases, AI feature stores) so the assessment can discover what is already out there — even if only as a provided inventory export (paths + basic metadata), never the payloads.

• **Why required:** Governance starts by finding the shadow assets already in circulation. The readiness assessment (Tool 1) profiles an inventory to show how many assets have no lineage, classification, or approval today.

• **How to verify:** Confirm you can obtain a metadata inventory (asset id/path, environment, approximate origin) of existing synthetic/test assets — read-only, metadata-only.

• **How to implement if missing:** Ask the platform/AI teams for an inventory export of their test data locations. If none exists, the assessment starts from the pilot's assets only and the expansion phase widens coverage.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate* ***synthetic*** *(artificial) data and run in* ***isolated non-production*** *test environments.* ***Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment*** *— where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls.* ***Synthetic-data and digital-twin tools*** *(e.g. Synthea, and commercial platforms) and* ***AI/model frameworks*** *are integrated at the pattern/architectural level; specific vendor console internals are* ***cited or deferred*** *to that vendor's documentation, never invented.* ***Regulatory and legal interpretation*** *(HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is* ***deferred to the organization's privacy, compliance, and legal functions*** *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only,

minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these steps in order. They realize the **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model, pilot-first, with explicit pass/fail gates and a defined rollback. Every artifact is **named** with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in later steps, testing, troubleshooting, and the scripts.

All governance registers are **JSON-backed, metadata-only** files stored in the `Governance Register Store – Enterprise – v1` (Requirement R4). They never hold asset payloads and never real PHI. The five Python tools are referenced by name; each is safe-by-default (dry-run where it writes) — see `08-deliverables-summary.md` for the full list.

Phase 0 — Confirm prerequisites

1. Verify the six category requirements (`global-requirements-synthetic.md`) and the five solution-specific requirements (`02-requirements.md`) are met. In particular confirm you have: `Declared Control Set – Enterprise – v1` (R1), `Classification Scheme – Enterprise – v1` (R2), `Governance Roles – Enterprise – v1` (R3), `Governance Register Store – Enterprise – v1` (R4), and an asset inventory hook (R5).

1. Confirm the **pilot scope**: one named pilot service line (e.g. `a pilot service line`) and the set of synthetic/test assets it uses. Everything in Phases 1–4 is scoped to this pilot.

Phase 1 — Assess (establish the readiness baseline)

1. Obtain a **metadata-only inventory** of existing synthetic/test assets (R5) as a JSON or CSV of asset id/path, environment, and any known origin. No payloads.

1. Run the **governance readiness assessment** tool (`assess_governance_readiness.py`) against that

inventory and against the presence/absence of policy, roles, classification scheme, and control set. It emits the **Synthetic Data Governance – Enterprise – Readiness** report: counts of assets with no lineage, no classification, and no approval, plus which governance foundations exist.

1. Review the readiness report with the data-governance and privacy stakeholders. This baseline is

the "before" evidence and sets the expansion backlog.

Gate G1 (Assess): a readiness baseline exists and has been reviewed. *Pass* = report produced and the shadow-asset count and foundation gaps are agreed. *Fail* = no inventory or no stakeholder review — stop and resolve before Design.

Phase 2 — Design (policy, scheme, roles, control mapping)

1. Author the **Synthetic Data Governance Policy – Enterprise – v1**: scope (what counts as a

governed synthetic/test asset), the roles from `Governance Roles – Enterprise – v1`, the classification scheme reference, and the rules for approval, access, retention, and disposal.

1. Finalize the **Classification Scheme – Enterprise – v1** (R2) with a handling rule per value

(allowed environments, access, retention). Keep it to 3–5 values.

1. Build the **Governance Control Map – Enterprise – v1**: for each control in

`Declared Control Set – Enterprise – v1` (R1), record which governance control this solution operates that maps to it, and the evidence artifact that demonstrates it (e.g. "approval records in the approval log"). Where no governance control maps, mark it `not-addressed` — do **not** claim alignment you cannot evidence. Interpretation of sufficiency is deferred to compliance.

1. Use the **scope/config helper** tool (`syngov_config.py`) to generate the starter register files

(empty catalog, empty approval log, and the control-map skeleton seeded from the declared control set) into the `Governance Register Store – Enterprise – v1`. Run it in `--dry-run` first to preview, then `--commit`.

Gate G2 (Design): policy, classification scheme, roles, and control map exist and are reviewed by privacy/compliance. *Pass* = all four artifacts approved. *Fail* = any missing or unreviewed — resolve before building.

Phase 3 — Generate/Build (catalog, classify, approve, access)

1. Stand up the **Synthetic Asset Catalog – Enterprise – v1** using the catalog/lineage register

tool (`asset_catalog_register.py`). For each **pilot** asset, register a record capturing: asset id, target model (FHIR/OMOP/native), environment, generator/source, **parent asset** (lineage), reference statistics used (if any), and steward. Run `--dry-run` to preview each write, then `--commit`.

1. **Classify** each pilot asset against `Classification Scheme – Enterprise – v1` using the

classification + approval tracker (`classify_approve_tracker.py classify ...`). Classification is a **stewarded decision** — the steward supplies the value; the tool records who, when, and why.

1. **Route approvals**: for each pilot asset a consumer needs, record an approval **request**

(requester, purpose, asset, target environment), then the reviewer's **decision** (approve / reject, with reviewer name and date) using `classify_approve_tracker.py request` and `... decide`. Requester and approver must differ (R3). Only assets with an `approved` record may move to a shared AI/QA environment.

1. **Apply the access model**: from the classification and approval records, produce the access

matrix (which classification is allowed in which environment for which consumer role) and apply it in the environment's existing access-control mechanism. The tools produce the *evidence*; the enforcement is your platform's IAM — verify against your platform's documentation.

Gate G3 (Build): every pilot asset is cataloged with lineage, classified, and — where consumed — approved, with access aligned. *Pass* = zero pilot assets missing lineage, classification, or (if consumed) an approval. *Fail* = any gap — do not proceed to Validate.

Phase 4 — Validate (run the governance gates on the pilot)

1. Run the **coverage/gate report** (`control_compliance_check.py coverage`) over the pilot catalog

and approval log. It reports, per pilot asset: lineage present, classification present, approval present (if consumed), and access aligned — with an overall pass/fail per gate.

1. Run the **policy/control compliance checker** (`control_compliance_check.py controls`) against

`Governance Control Map – Enterprise – v1` and `Declared Control Set – Enterprise – v1`. It emits an evidence view: each declared control marked `in-place`, `partial`, or `not-addressed`, with the mapped governance control and evidence artifact. **This is evidence for compliance to interpret — it is not a compliance determination.**

1. Review both outputs with privacy/compliance. Record the review as the pilot's validation

evidence.

Gate G4 (Validate): the pilot passes the governance gates (lineage/classification/approval/ access) and the control evidence view has been reviewed. *Pass* = coverage report all-pass on the pilot and control evidence reviewed. *Fail* = any gate red — **roll back**: pull un-approved or misclassified assets from shared environments, hold the release, resolve the finding, and re-run Phase 4.

Phase 5 — Expand (widen coverage, carry the gates)

1. With the pilot passed, extend the **Synthetic Asset Catalog – Enterprise – v1**, classification,

and approval workflow to the next service line and asset type from the expansion backlog (Step 5). Reuse the same tools and the same gates (G3, G4) for each new scope.

1. Re-run the readiness assessment (`assess_governance_readiness.py`) periodically against the full

inventory to show the shadow-asset count trending down — the durable measure that governance is winning.

1. Keep the registers current: new asset ⇒ catalog + classify + approve before shared use; retired asset ⇒ mark disposed per the policy's retention rule. Governance is continuous, not a one-time project.

Rollback (applies at every gate)

If any gate fails, the release is **held**. Un-approved or misclassified assets are removed from shared AI/QA environments, the finding is recorded against the asset in the catalog, and the phase is re-run once resolved. No governed asset reaches a shared environment on a failed gate.

Testing & Validation

These procedures validate that synthetic data governance is actually operating — not just documented. They map to the gates in `03-implementation.md` and use the five delivered Python tools. Each test states its **expected pass/fail** signal. Run them on the named pilot before Expand, and re-run the relevant ones whenever the registers change.

> ***Boundary reminder.*** *These tests validate the* ***governance method and its evidence*** *— completeness of lineage, classification, approval, access, and control mapping. They do* ***not*** *determine HIPAA/HITRUST compliance, de-identification adequacy, or disclosure-risk acceptability; those remain with your privacy, compliance, and legal functions.*

T1 — Readiness baseline is produced (Gate G1)

- **Procedure:** Run `assess_governance_readiness.py` against the metadata-only asset inventory and

the governance-foundation flags (policy/roles/scheme/control-set present or not). Confirm it emits the **Synthetic Data Governance – Enterprise – Readiness** report.

- **Expected pass:** A report is produced showing counts of assets with no lineage, no

classification, and no approval, plus which foundations exist. Stakeholders have reviewed it.

- **Fail:** No inventory available, tool errors, or no review. Resolve before Design.

T2 — Design artifacts exist and are reviewed (Gate G2)

- **Procedure:** Confirm `Synthetic Data Governance Policy – Enterprise – v1`,

`Classification Scheme – Enterprise – v1`, `Governance Roles – Enterprise – v1`, and `Governance Control Map – Enterprise – v1` all exist and carry a privacy/compliance review note. Confirm `syngov_config.py --commit` created the starter registers in the store.

- **Expected pass:** All four artifacts present and reviewed; the empty catalog, empty approval log,

and control-map skeleton exist in `Governance Register Store – Enterprise – v1`.

- **Fail:** Any artifact missing or unreviewed, or the starter registers absent.

T3 — Every pilot asset has lineage (catalog completeness)

- **Procedure:** Run `control_compliance_check.py coverage` over `Synthetic Asset Catalog – Enterprise – v1`

restricted to the pilot. Inspect the `lineage` column: each pilot asset must record a generator/source and, where derived, a `parent asset`.

- **Expected pass:** 100% of pilot assets have a recorded origin; derived assets record a parent.

The report's lineage gate = **PASS**.

- **Fail:** Any pilot asset with no recorded origin. Fix in the catalog, re-run.

T4 — Every pilot asset is classified (classification completeness)

- **Procedure:** In the same coverage report, inspect `classification`. Each value must be one of

`Classification Scheme – Enterprise – v1` and carry a stewarded record (who/when/why) in the classification log.

- **Expected pass:** 100% of pilot assets classified with a valid scheme value and a stewarded

record. Classification gate = **PASS**.

- **Fail:** Any asset unclassified or carrying a value outside the scheme.

T5 — Approval is enforced before shared use (approval integrity)

- **Procedure:** For each pilot asset that a consumer uses in a shared AI/QA environment, confirm the

approval log has an `approved` decision with a **requester** ≠ **approver**, a purpose, the target environment, a named reviewer, and a date. Attempt (in dry-run) to mark an asset consumable that has no approval and confirm the coverage report flags it.

- **Expected pass:** Every consumed pilot asset has a valid `approved` record; any consumed asset

lacking one is flagged red. Approval gate = **PASS**. Requester and approver are never the same person.

- **Fail:** A consumed asset with no approval, a self-approval, or an approval missing required

fields.

T6 — Access is aligned to classification (access model)

- **Procedure:** Compare the access matrix produced from classification+approval against the actual

environment access configuration. Confirm no asset of a higher classification is reachable in a lower-controlled environment than its handling rule allows.

- **Expected pass:** Every asset's effective access matches its classification's handling rule; no

higher-classification asset in a disallowed environment. Access gate = **PASS**.

- **Fail:** Any misaligned access. Roll back the asset from the environment and correct.

T7 — Control evidence view is complete and honest (control mapping)

• **Procedure:** Run `control_compliance_check.py` against `Governance Control Map – Enterprise – v1`

and `Declared Control Set – Enterprise – v1`. Confirm every declared control resolves to `in-place`, `partial`, or `not-addressed`, each with a mapped governance control and an evidence artifact reference (or an explicit `none`).

• **Expected pass:** No declared control is left unresolved; `in-place` controls each cite an

evidence artifact that actually exists in the registers; unaddressed controls are honestly marked `not-addressed` (not silently claimed). The output is labeled **evidence for compliance to interpret**.

• **Fail:** A declared control with no status, an `in-place` claim with no backing evidence, or a

gap hidden as addressed.

T8 — Rollback works

• **Procedure:** Simulate a failed gate: mark a pilot asset misclassified (dry-run) and confirm the

documented rollback pulls it from the shared environment list, records the finding against the asset, and holds the release until resolved.

• **Expected pass:** The asset is removed from shared-environment eligibility, the finding is

recorded, and the coverage report shows the release held.

• **Fail:** The asset remains eligible after a failed gate, or no finding is recorded.

Overall pilot pass criteria

The pilot **passes** and may proceed to Expand only when T1–T8 all pass: readiness baselined, design artifacts reviewed, and the pilot catalog shows **100% lineage, 100% classification, enforced approval, aligned access**, with a complete, honest control-evidence view reviewed by privacy/compliance and a demonstrated rollback.

Troubleshooting

Common failure modes when standing up and operating synthetic data governance, with resolutions. Symptoms are grouped by phase. All tools are safe-by-default (dry-run where they write); when in doubt, re-run without `--commit` to preview.

> ***Boundary reminder.*** *These resolutions fix the governance **method and evidence**. They never substitute for a privacy/compliance/legal determination on HIPAA/HITRUST de-identification, disclosure risk, or control sufficiency.*

Assess

Symptom: `assess_governance_readiness.py` **reports 0 assets.**

• *Cause:* the inventory JSON is empty, or the assets are under a key the tool does not read.

• *Resolution:* the tool accepts a bare list or `{"assets": [...]}`. Confirm the inventory has an `assets` array (or is a top-level list) with one object per asset. It is metadata-only — no payloads needed.

Symptom: everything shows as "no lineage" even for assets you know the origin of.

• *Cause:* lineage is read from an asset's `generator` or `source` field; the inventory export uses a different field name.

• *Resolution:* map the export to include `generator` (or `source`) per asset before assessing. The assessment's whole point is to surface the shadow assets that genuinely lack a recorded origin — don't mask real gaps, but do fix field-name mismatches.

Design

Symptom: `syngov_config.py validate` **fails "scheme has N values (keep it to 3-5)".**

• *Cause:* the classification scheme is too large or too small.

• *Resolution:* collapse the scheme to 3–5 values with clear handling rules. A sprawling taxonomy becomes inconsistent in practice (see Best Practices). If you genuinely need more, that is a policy decision — but validate the tradeoff first.

Symptom: `syngov_config.py validate` **fails a classification "has no handling_rule".**

• *Cause:* a scheme value lacks a `handling_rule` (allowed environments/access/retention).

• *Resolution:* add a handling rule to every value. Classification without a handling rule cannot drive access, which defeats its purpose.

Symptom: `syngov_config.py scaffold` **says** `[skip]` **for a file you wanted created.**

• *Cause:* the register already exists — scaffold never overwrites, by design (it must not clobber a live catalog or approval log).

• *Resolution:* this is expected and safe. To re-seed a register, move the existing file aside first, then re-run scaffold.

Symptom: the control map is seeded but every control is `not-addressed`**.**

• *Cause:* that is the honest default — scaffold seeds from the declared control set with no mappings claimed.

• *Resolution:* the steward fills in `governance_control` and `evidence` and sets `status` to `in-place`/`partial` only where a real governance control and evidence artifact exist. Never mass- set to `in-place` to "look done"; the compliance checker will flag `in-place` with no evidence.

Build

Symptom: `asset_catalog_register.py` **errors "an asset needs an origin".**

• *Cause:* neither `--generator` nor `--source` was supplied — the asset would have no lineage.

• *Resolution:* supply the generating tool/pattern (`--generator`) or the source system

(`--source`). An asset with no origin is a finding, so the tool refuses to register one.

Symptom: registering a derived asset warns "parent … is not in the catalog".

- *Cause:* the `--parent` id has not been registered yet — lineage would be broken.
- *Resolution:* register the parent asset first, then the child. Lineage is the spine; a dangling parent reference is exactly what governance exists to prevent.

Symptom: `classify_approve_tracker.py classify` **rejects the value.**

- *Cause:* the value is not in `Classification Scheme – Enterprise – v1`.
- *Resolution:* use an agreed scheme value. If the asset genuinely needs a new category, that is a scheme change (a stewarded/policy decision), not a per-asset override.

Symptom: `decide` **errors "requester and approver must differ".**

- *Cause:* the reviewer is the same person who requested the asset — a self-approval.
- *Resolution:* route the decision to a different reviewer (separation of duties, R3). This guard is intentional and should not be worked around.

Symptom: `decide` **errors "no open approval request".**

- *Cause:* no `request` was recorded for that asset (or the last one is already decided).
- *Resolution:* run `classify_approve_tracker.py request …` first, then `decide`.

Validate

Symptom: `control_compliance_check.py coverage` **shows an asset FAIL on approval though it looks approved.**

- *Cause:* the asset is marked `consumed_in_shared` but has no `approved` decision, or the approval's `target_env` does not match the asset's environment.
- *Resolution:* record (or complete) the approval for the correct target environment. Approval is required only for assets consumed in a shared environment — if it is not actually consumed there, unset `--consumed-in-shared` on the catalog record.

Symptom: coverage shows an access FAIL.

- *Cause:* a consumed asset has no approval matching its environment — access is not evidenced as aligned.
- *Resolution:* approve the asset for that environment, or (rollback) pull it from the shared environment until approved. Then re-run coverage.

Symptom: `controls` **reports "in-place w/o evidence".**

- *Cause:* a mapping claims `status: in-place` but has no `evidence` artifact.
- *Resolution:* add the evidence artifact reference (e.g. "approval records in the approval log") or

downgrade the status to `partial`/`not-addressed`. An unbacked `in-place` claim will not survive an audit — honesty here is the whole value.

Symptom: `controls` **reports "UNRESOLVED" for a declared control.**

- *Cause:* the declared control has no entry in the control map (id mismatch, or missing mapping).
- *Resolution:* add a mapping for that control id (even if `not-addressed`). Every declared control must resolve to a status; a missing one is a hole in the evidence view.

Operate / Expand

Symptom: the shadow-asset count is not falling across readiness runs.

- *Cause:* new assets are entering shared environments without being cataloged/classified/approved — the governance gate is being bypassed.
- *Resolution:* enforce "catalog + classify + approve before shared use" at the platform level (access model), not just in policy. Investigate how ungoverned assets are reaching shared environments and close that path.

Symptom: approvals route to someone who has left the team.

- *Cause:* `Governance Roles – Enterprise – v1` is stale after a reorg.
- *Resolution:* re-confirm role holders on a set cadence and update the roles config. Validate again with `syngov_config.py validate`.

Best Practices

Durable practices for operating synthetic data governance in a regulated healthcare enterprise.

Governance discipline

- **Lineage is the spine — an asset with no origin is a finding.** Never let a synthetic/test asset enter a shared environment without a catalog record naming its generator/source and, if derived, its parent asset. "We're not sure where this came from" is exactly the shadow risk this solution exists to kill.
- **Classify before you share, not after.** Classification drives access and the controls that apply. An unclassified asset is ungoverned by definition; block it from shared use until a steward records a classification value from `Classification Scheme – Enterprise – v1`.
- **Approval is a recorded human decision, never a default.** Every release to a shared AI/QA environment carries an approval record with a distinct requester and approver, a purpose, and a date. Self-approval is not approval.
- **Keep the classification scheme small.** 3–5 values with clear handling rules beat a sprawling taxonomy nobody applies consistently. Complexity in the scheme becomes inconsistency in practice.

Keep the boundary bright

- **Deliver method and evidence — defer the determination.** Produce the catalog, classification, approval, and control-evidence artifacts; let privacy/compliance/legal decide HIPAA/HITRUST de-identification adequacy, disclosure-risk acceptance, and control sufficiency. Never let a tool's green output be read as a compliance sign-off.
- **The declared control set is authoritative — do not edit it to look better.** Map honestly. Mark unaddressed controls `not-addressed`; a truthful gap is more valuable than a hidden one, and an overstated `in-place` will fail an audit.
- **Never real PHI into a shared harness/twin/AI test environment.** Governance records are metadata-only. Where real data must inform fidelity, it is read-only, minimized, and under the privacy controls — and the asset that touched it carries the `contains-minimized-real` (or your scheme's equivalent) classification with the stricter handling rule.

Registers & evidence

- **Metadata-only, version-controlled, access-controlled.** The registers hold no payloads and no PHI. Store them where changes are tracked and approval records cannot be silently altered — the audit trail *is* the value.
- **One source of truth per register.** Catalog, classification/approval log, and control map are the authoritative copies. Integrate with an enterprise data catalog if one exists rather than forking a second truth.
- **Dry-run first, commit second.** Every tool that writes defaults to dry-run. Preview the change, confirm it, then `--commit`. This is habit, not ceremony — it prevents a fat-fingered approval.

Process & scale

- **Pilot-first, always.** Prove the full gate set (lineage → classification → approval → access → control evidence) on one service line before expanding. A pattern that passes a pilot scales; one that skips the pilot multiplies unproven risk.
- **Watch the shadow-asset count trend.** Re-run the readiness assessment periodically against the full inventory. A falling count of assets-with-no-lineage/classification/approval is the durable proof governance is working; a rising one is an early warning.
- **Requester ≠ approver, enforced.** Bake separation of duties into the approval tool and the reviewers' expectations, not just the policy PDF.
- **Retire assets on purpose.** Apply the policy's retention/disposal rule; mark disposed assets in the catalog. An accumulating pile of stale "test" datasets is future shadow risk.
- **Review roles when the org changes.** Owner/steward/reviewer/consumer assignments drift with

reorgs. Re-confirm `Governance Roles – Enterprise – v1` on a set cadence so approvals never route to someone who has moved on.

Deliverables Summary

What the buyer gets — a complete, self-guided package to establish synthetic data governance for regulated healthcare, aligned to HIPAA/HITRUST + enterprise data governance. Regulatory interpretation is deferred to your privacy, compliance, and legal functions; this package delivers the governance **method and evidence**.

Documentation

• **Solution Overview** (`01-solution-overview.md`) — the six-component governance model (policy &

roles, catalog & lineage, classification, approval workflow, access model, controls & alignment) and the Assess → Design → Generate/Build → Validate → Expand deployment model.

• **Requirements Guide** (`02-requirements.md`) — the solution-specific prerequisites (declared

control set, classification scheme, governance roles, register store, inventory hook), each with how to verify and how to implement if missing, on top of the six category-wide requirements.

• **Implementation Guide** (`03-implementation.md`) — exact, ordered steps through all five phases,

every artifact named, with the four pass/fail gates and a rollback.

• **Testing & Validation** (`05-testing.md`) — eight procedures (T1–T8) mapping to the gates, each

with an explicit pass/fail signal, and the overall pilot pass criteria.

• **Troubleshooting** (`06-troubleshooting.md`) — common failure modes and resolutions.

• **Best Practices** (`07-best-practices.md`) — durable governance, boundary, register, and scale

practices.

• **Executive Summary** (`executive-summary.md`) — the outcome, approach, and why it is safe.

The 5 Python tools

All tools are Python 3.10+ standard-library-first, safe-by-default (dry-run where they write, never real PHI, no invented network calls), and self-documenting via `--help`. They read and write the JSON-backed, metadata-only governance registers.

1. `assess_governance_readiness.py` — *Governance readiness assessment.* Profiles a metadata-only

asset inventory plus the governance-foundation flags and emits the **Synthetic Data Governance – Enterprise – Readiness** report: how many assets have no lineage, no classification, and no approval, and which foundations (policy/roles/scheme/control-set) exist. The "before" baseline and the trend metric.

1. `asset_catalog_register.py` — *Synthetic-asset catalog & lineage register.* Registers each

governed synthetic/test asset into the **Synthetic Asset Catalog – Enterprise – v1** (JSON-backed) with id, target model, environment, generator/source, **parent asset** (lineage), reference stats used, and steward. Dry-run by default; `--commit` to write. Lists and queries the catalog.

1. `classify_approve_tracker.py` — *Classification & approval tracker.* Records the stewarded

classification of each asset (against the scheme) and drives the approval workflow — `request` (requester/purpose/target env), `decide` (approve/reject, reviewer, date), enforcing requester ≠ approver — into the JSON classification/approval log. Dry-run by default; `--commit` to write.

1. `control_compliance_check.py` — *Policy/control compliance checker.* Maps the governance

controls this solution operates against the org's **declared control set**, marking each control `in-place`, `partial`, or `not-addressed` with the mapped control and evidence artifact. Emits **evidence for compliance to interpret — not a compliance determination.** (Doubles as the governance coverage/gate report over catalog + approval log.)

1. `syngov_config.py` — *Scope & config helper.* Generates the starter register files (empty

catalog, empty classification/approval log, control-map skeleton seeded from the declared control set), validates the classification scheme and roles config, and scaffolds the `Governance Register Store – Enterprise – v1` layout. Dry-run by default; `--commit` to write.

Register artifacts (you produce these)

- `Synthetic Data Governance Policy – Enterprise – v1`
- `Classification Scheme – Enterprise – v1`
- `Governance Roles – Enterprise – v1`
- `Declared Control Set – Enterprise – v1` (input from compliance)
- `Governance Control Map – Enterprise – v1`
- `Synthetic Asset Catalog – Enterprise – v1`
- `Synthetic Data Governance – Enterprise – Readiness` (the assessment report)

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

assess_governance_readiness.py

asset_catalog_register.py

classify_approve_tracker.py

control_compliance_check.py

syngov_config.py